Category:

Network

Name:

SMB3overKerberos

Message:

We've been under attack! The network monitoring team reports that the adversaries were using Impacket. A packet capture from the incident is provided: impacket-attack-captured.pcap.

Analyze the traffic and find the flag file. Let's hope it hasn't been stolen…

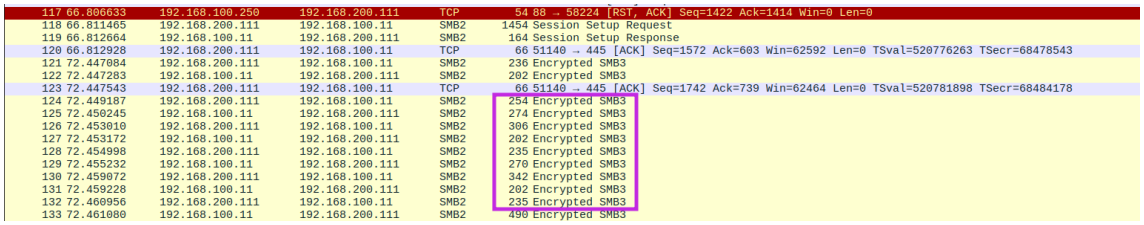
To proof your understanding, please create flag by "**CSG_FLAG{Nothing_escapes_my_sight!}**"

Instructions:

First, we need to grasp the overall attack flow and consider our analysis strategy. Encrypted SMB3 communication is immediately noticeable.

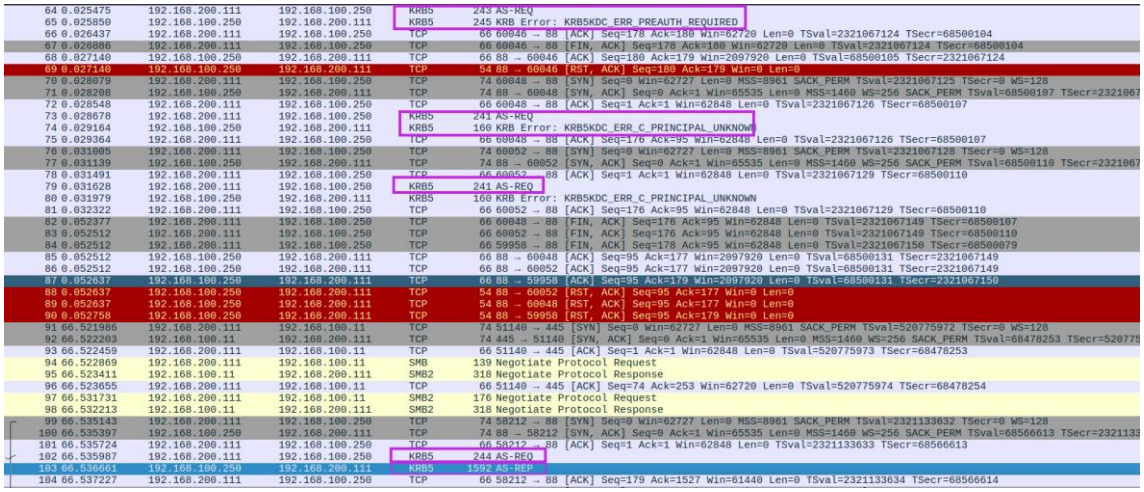
This refers to encrypted communication within an SMB session. Using Wireshark's capabilities, we can decrypt the communication content if we have the service session key and service session ID. The following link provides more details on how to do this:

https://wiki.samba.org/index.php/Wireshark_Decryption



117	66.806633	192.168.100.250	192.168.200.111	TCP	54	88 → 58224 [RST, ACK] Seq=1422 Ack=1414 Win=0 Len=0
118	66.811465	192.168.200.111	192.168.100.11	SMB2	1454	Session Setup Request
119	66.812664	192.168.100.11	192.168.200.111	SMB2	164	Session Setup Response
120	66.812928	192.168.200.111	192.168.100.11	TCP	66	51140 → 445 [ACK] Seq=1572 Ack=603 Win=62592 Len=0 TSval=520776263 TSecr=68478543
121	72.447084	192.168.200.111	192.168.100.11	SMB2	236	Encrypted SMB3
122	72.447283	192.168.100.11	192.168.200.111	SMB2	202	Encrypted SMB3
123	72.447543	192.168.200.111	192.168.100.11	TCP	66	51140 → 445 [ACK] Seq=1742 Ack=739 Win=62464 Len=0 TSval=520781898 TSecr=68484178
124	72.449187	192.168.200.111	192.168.100.11	SMB2	254	Encrypted SMB3
125	72.450245	192.168.100.11	192.168.200.111	SMB2	274	Encrypted SMB3
126	72.453010	192.168.200.111	192.168.100.11	SMB2	306	Encrypted SMB3
127	72.453172	192.168.100.11	192.168.200.111	SMB2	202	Encrypted SMB3
128	72.454998	192.168.200.111	192.168.100.11	SMB2	235	Encrypted SMB3
129	72.455232	192.168.100.11	192.168.200.111	SMB2	270	Encrypted SMB3
130	72.459072	192.168.200.111	192.168.100.11	SMB2	342	Encrypted SMB3
131	72.459228	192.168.100.11	192.168.200.111	SMB2	202	Encrypted SMB3
132	72.460956	192.168.200.111	192.168.100.11	SMB2	235	Encrypted SMB3
133	72.461080	192.168.100.11	192.168.200.111	SMB2	490	Encrypted SMB3

Additionally, the multiple AS-REQ requests are suspicious. This could indicate an AS-REP Roasting attack.



Based on the analysis so far, our overall strategy is to decrypt the SMB3 communication by obtaining the service session key and service session ID, and to conduct a detailed analysis of the AS-REQ related communications.

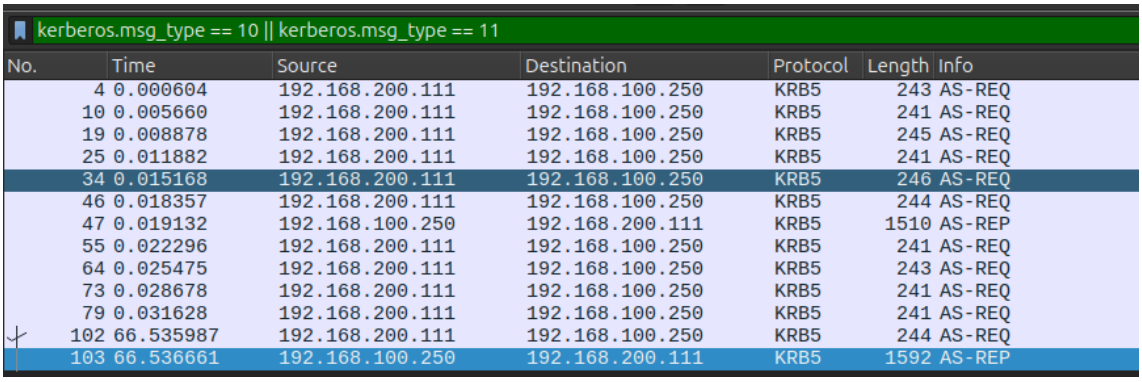
To obtain the service session key and service session ID, we need to analyze the TGT and TGS communications in environments where Kerberos authentication is used. The following blog describes the communication procedure in a Kerberos authentication environment. Briefly, the process is as follows:

https://blog.redforce.io/windows-authentication-attacks-part-2-kerberos.

1. The client requests a TGT from the Authentication Server (AS) with an AS-REQ.
2. The AS returns a message containing the TGT to the client in an AS-REP.
3. The client saves the obtained TGT in the LSASS memory of the terminal.
4. Because the cached TGT in memory proves that the client has been authenticated, the client sends a TGS-REQ to the server, including the information of the service to be used.
5. The server returns data including the service session key to the client in a TGS-REP.
6. The client uses the service session key and service session ID obtained from the TGS-REP to access the desired service (e.g., SMB).

AS-REQ/AS-REP Analysis

1. To make the AS-REQ and AS-REP communication easier to view, we can use a filter in Wireshark. This reveals that multiple AS-REQ requests are sent from the client, but only two are successful.



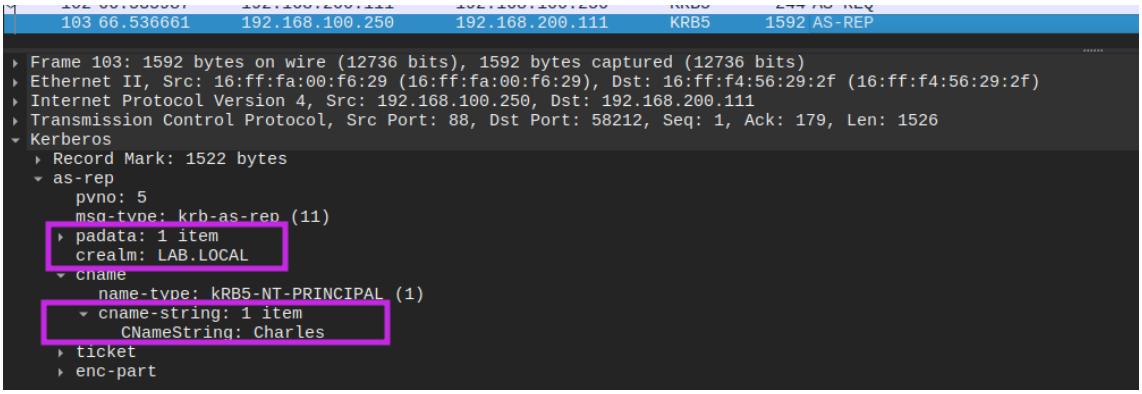
   Also, the AS-REQ and AS-REP exchange is the communication that the client performs with the AS to obtain a TGT in an environment that requires Kerberos authentication. More details on Kerberos V5 can be found in RFC 4120: https://datatracker.ietf.org/doc/html/rfc4120

2. RFC 4120 describes that if pre-authentication is not enabled for the requested principal in the AS-REQ, the AS returns an AS-REP without verifying that the client is indeed that principal. It also mentions that the user receiving the AS-REP may be able to obtain the password of that principal from the obtained AS-REP message. This is an attack technique called AS-REP Roasting, which is also mentioned in section 3.1 of RFC 4120. *3.1 The Authentication Service Exchange*

3. Analyzing the AS-REP communication, we can find the username in Kerberos > as-rep > cname > cname-string > CNameString and the domain name in Kerberos > as-rep > crealm.

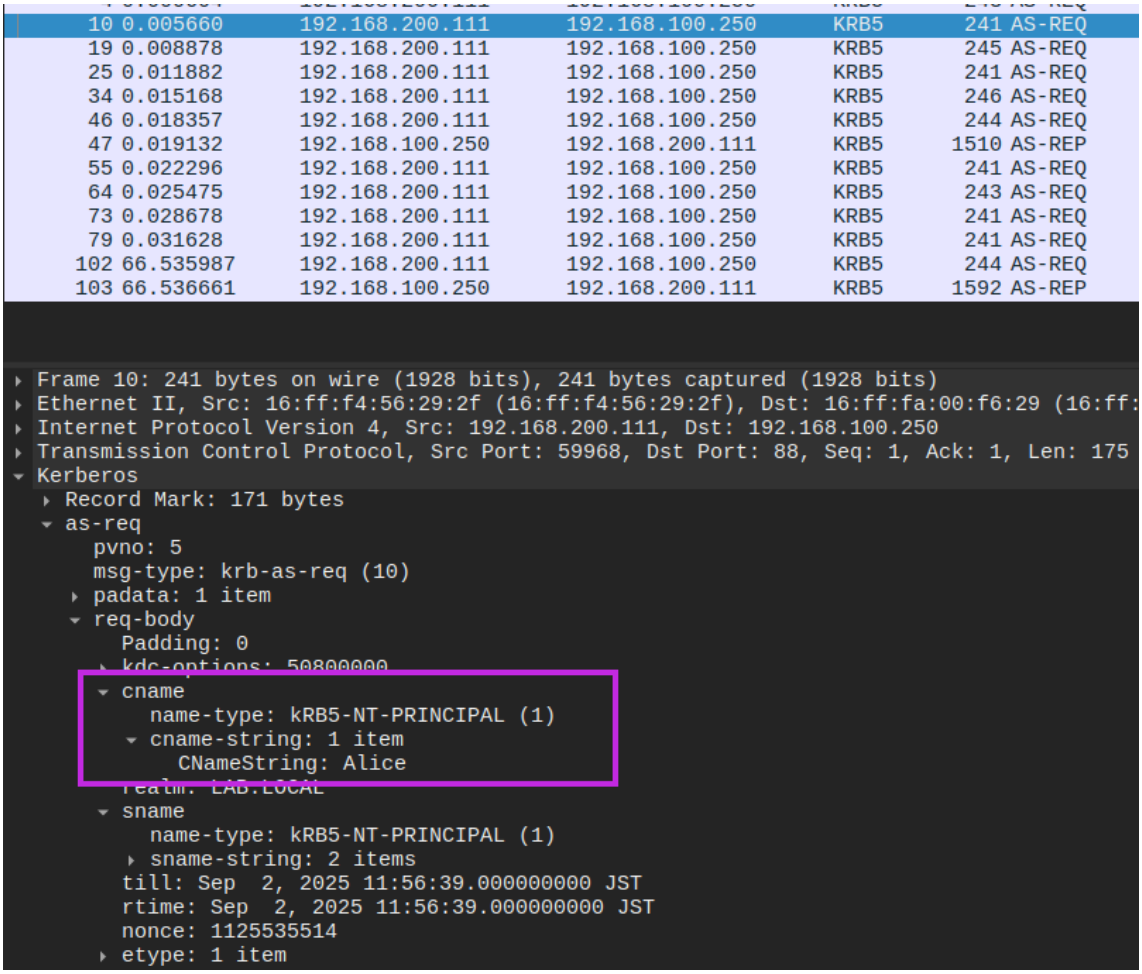
   CnameString: Charles

   crealm: LAB.LOCAL



4. Normally, if pre-authentication is enabled for the AS-REQ principal, the client receives an

error message **KRB5KDC_ERR_PREAUTH_REQUIRED** and no AS-REP is returned. However, no error is returned for Charles@LAB.LOCAL, for which an AS-REP was returned. From this, we can infer that pre-authentication is not enabled for Charles@LAB.LOCAL. The AS-REP Roasting attack allows an attacker to obtain the hash value of a domain user's password without knowing the actual password. It's possible that the attacker cracked and obtained Charles's password from this AS-REP.



5. There are tools to parse the values obtained from the AS-REP and convert them into a format that can be cracked by John the Ripper or Hashcat. One such tool is the impacket tool. Looking at the implementation of this tool, we can see that the AS-REP is converted into the following format for cracking:

   $krb5asrep$@:$<first 32 characters of cipher>$<characters after the 33rd character of cipher>.

   By assembling the values and cracking them with John, we can obtain the value **Passw0rd!**.

```
176             # Let's output the TGT enc-part/cipher in John format, in case somebody wants to use it.
177             if self.__outputFormat == 'john':
178                 # Check what type of encryption is used for the enc-part data
179                 # This will inform how the hash output needs to be formatted
180                 if asRep['enc-part']['etype'] == 17 or asRep['enc-part']['etype'] == 18:
181                     return '$krb5asrep$%d$%s%s$%s$%s' % (asRep['enc-part']['etype'], domain, clientName,
182                                                          hexlify(asRep['enc-part']['cipher'].asOctets()[:-12]).decode(),
183                                                          hexlify(asRep['enc-part']['cipher'].asOctets()[-12:]).decode())
184                 else:
185                     return '$krb5asrep$%s@%s:%s$%s' % (clientName, domain,
186                                                        hexlify(asRep['enc-part']['cipher'].asOctets()[:16]).decode(),
187                                                        hexlify(asRep['enc-part']['cipher'].asOctets()[16:]).decode())
```

```
[redacted] john --format=krb5asrep hash.txt -w=./wordlist.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5asrep, Kerberos 5 AS-REP etype 17/18/23 [MD4 HMAC-MD5 RC4 / PBKDF2 HMAC-SHA1 AES 256/256 AVX2 8x])
Cost 1 (etype) is 23 for all loaded hashes
Will run 12 OpenMP threads
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
Warning: Only 3 candidates buffered, minimum 96 needed for performance.
Passw0rd!        ($krb5asrep$chales@lab.local)
1g 0:00:00:00 DONE (2025-09-19 10:03) 33.33g/s 100.0p/s 100.0c/s 100.0C/s Passw0rd!..Passw0rd1!
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

The following link provides more details:
https://github.com/fortra/impacket/blob/master/examples/GetNPUsers.py#L184

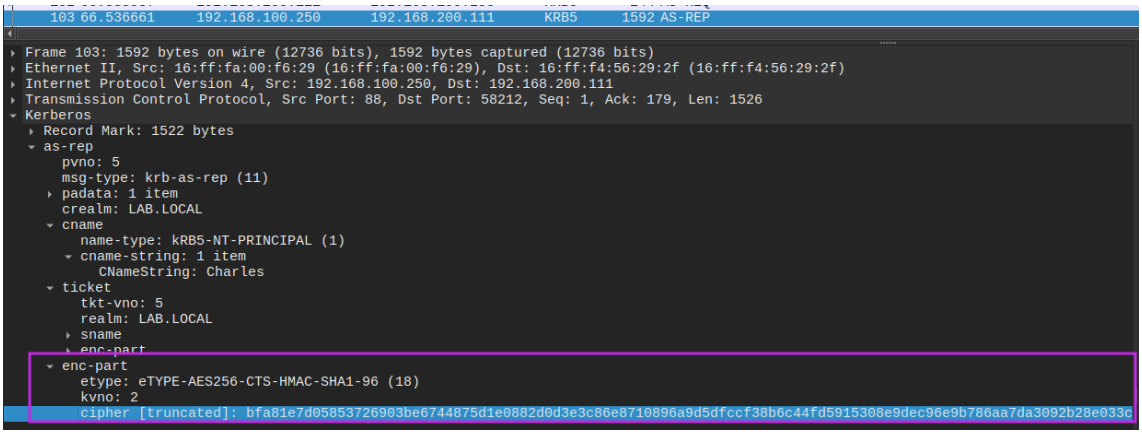
The cipher from packet No. 47 uses RC4 encryption. It is also confirmed that AS-REQ is performed for other users, but pre-authentication is enabled for these users, and an error is returned immediately after AS-REQ.

```
    47 0.019132    192.168.100.250    192.168.200.111    KRB5    1510 AS-REP
    55 0.022296    192.168.200.111    192.168.100.250    KRB5     241 AS-REQ
    64 0.025475    192.168.200.111    192.168.100.250    KRB5     243 AS-REQ
    73 0.028678    192.168.200.111    192.168.100.250    KRB5     241 AS-REQ
    79 0.031628    192.168.200.111    192.168.100.250    KRB5     241 AS-REQ
   102 66.535987   192.168.200.111    192.168.100.250    KRB5     244 AS-REQ
   103 66.536661   192.168.100.250    192.168.200.111    KRB5    1592 AS-REP

▸ Frame 47: 1510 bytes on wire (12080 bits), 1510 bytes captured (12080 bits)
▸ Ethernet II, Src: 16:ff:fa:00:f6:29 (16:ff:fa:00:f6:29), Dst: 16:ff:f4:56:29:2f (16:ff:f4:56:29:2f)
▸ Internet Protocol Version 4, Src: 192.168.100.250, Dst: 192.168.200.111
▸ Transmission Control Protocol, Src Port: 88, Dst Port: 60022, Seq: 1, Ack: 179, Len: 1444
▾ Kerberos
  ▸ Record Mark: 1440 bytes
  ▾ as-rep
      pvno: 5
      msg-type: krb-as-rep (11)
      crealm: LAB.LOCAL
    ▾ cname
        name-type: kRB5-NT-PRINCIPAL (1)
      ▾ cname-string: 1 item
          CNameString: Charles
    ▸ ticket
    ▾ enc-part
        etype: eTYPE-ARCFOUR-HMAC-MD5 (23)
        kvno: 2
        cipher [truncated]: 2d44fb28479b60f045a20a137fae8525a22e8452f830f5cde20bad4872d024dbb3d90c128046c572a9657410b9775607ed1ddf
```

Extracting session key from TGT

1. The AS-REP contains the session key, which is used in the authorization process after TGS. Checking packet No. 103 in Wireshark, the session key is included in the enc-part.cipher data obtained from the AS-REP.



2. This cipher is encrypted with AES256, as defined in enc-part.etype. To decrypt and extract the session key, we need to decrypt the cipher. Sections 4 of RFC 3962 and the following blog provide details on this encryption:
   https://datatracker.ietf.org/doc/html/rfc3962
   https://blog.redforce.io/windows-authentication-attacks-part-2-kerberos

   Briefly, a long-term key is generated based on the user's password, realm, and username information, and then the enc-part.cipher value is decrypted using the long-term key.
3. It is convenient to use impacket to implement the above decryption process. Below is a PoC for extracting the session key from the AS-REP based on Charles's password.

```
from binascii import hexlify
from impacket.krb5.crypto import string_to_key, _enctype_table, Key
from binascii import unhexlify
from pyasn1.codec.der import decoder

etype = _enctype_table[18]

passphrase = "Passw0rd!"
realm = "LAB.LOCAL"
username = "Charles"
salt = realm + username
```

```
long_term_key = string_to_key(enctype=18, string=passphrase,
salt=salt)
long_term_key = hexlify(long_term_key.contents).decode()
#print("Derived long-term key:",
hexlify(long_term_key.contents).decode())

key = Key(18, unhexlify(long_term_key))
mycipher =
"bfa81e7d05853726903be6744875d1e0882d0d3e3c86e8710896a9d5dfccf38b6c4
4fd5915308e9dec96e9b786aa7da3092b28e033cc6fec883bfaa0fcf4d8fe71eb4e7
144ca85e92e084a4bc7d51f75446bda880a21f52117af13a3628c481ca0fa7deef66
45f8ec75566250fce5eb4694537aa3ce35c6c331a9fbb377e639c9212cb506c6d772
9964e1b371df5c5ce364a6a68d52484e882b2436d1183c2a8879ba0bd72611998f30
b6514f0af2f64a8d60ae5be369d5c223ecf79288689cbd16f4c297c2cf3775a9873f
6b7326d9b3590ec68b3b7db41a9e1ab9d37f3ccb8af0e8bf5aeb67c531aefd28f3b2
db7729b8400b33a286616a4c597930fcf1d66afd82e5f26ca43912868"

jnk = etype.decrypt(key, 3, unhexlify(mycipher))
dec = decoder.decode(jnk)
for i in dec:
  print(i)
```

Also, refer to the following for the implementation of impacket:

Derivation of the long-term key:

https://github.com/fortra/impacket/blob/master/impacket/krb5/crypto.py#L447

Decryption logic of enc-part.cipher using the long-term key:

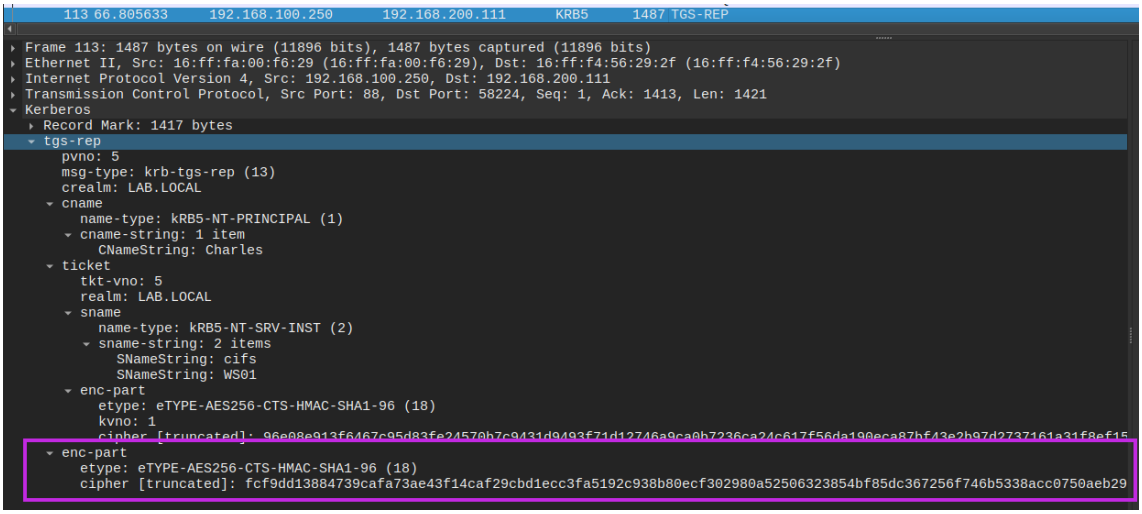
https://github.com/fortra/impacket/blob/master/impacket/krb5/crypto.py#L540

4. Running the above PoC, we can obtain the following session key:
09325be3bd041fe8a26024d029b46c80376ed00c42a5e54611d4126e109a4e7d.

```
Sequence:
 field-0=Sequence:
  field-0=18
  field-1=0x09325be3bd041fe8a26024d029b46c80376ed00c42a5e54611d4126e109a4e7d
```

Extracting the Service Session Key from the TGS

1. The session key included in the TGS is described in section 5.4.2 of RFC 4120: KRB_KDC_REP Definition: https://tex2e.github.io/rfc-translater/html/rfc4120.html
2. To check in Wireshark, it is described in the enc-part.cipher of TGS-REP in No. 113.



3. The above cipher can be decrypted using the session key extracted from AS-REP, using the following PoC. We also use impacket for this decryption.

```
from binascii import hexlify
from impacket.krb5.crypto import _enctype_table, Key
from binascii import unhexlify
from pyasn1.codec.der import decoder
from impacket.krb5.asn1 import EncTGSRepPart

tgs_cipher =
"fcf9dd13884739cafa73ae43f14caf29cbd1ecc3fa5192c938b80ecf302980a5250
6323854bf85dc367256f746b5338acc0750aeb29215bfd7fd273df8cd170f964f301
eec37e98754cb91c8bc669ccac419ec664fca8e295c1cdc66177e8ff734d94a667e0
f7f1b283bed16641ea8742b067b59a0072664d1cb5d3ca8ffe69c994d0638c2ad40d
116252a567c33d2c516b8d5be59b0b7528340e8b1f258ea5db054a73527fe4cf8ccb
403e5c2cef7152e34a78057cc8f7483ca56b6ee6a87c9118e37062436af4f2994258
259f4eed17f507dd17739afb743839efdc8aed2b616a390b3a262f64611a9c8680e3
456579a89ff23"
tgt_session_key =
"09325be3bd041fe8a26024d029b46c80376ed00c42a5e54611d4126e109a4e7"
```

```
etype = 18

ciphertext = unhexlify(tgs_cipher)
key = Key(etype, unhexlify(tgt_session_key))

plaintext = _enctype_table[etype].decrypt(key, 8, ciphertext)

decoded, _ = decoder.decode(plaintext, asn1Spec=EncTGSRepPart())

svc_key = decoded['key']
print("service session key:",
hexlify(bytes(svc_key['keyvalue'])).decode())
```

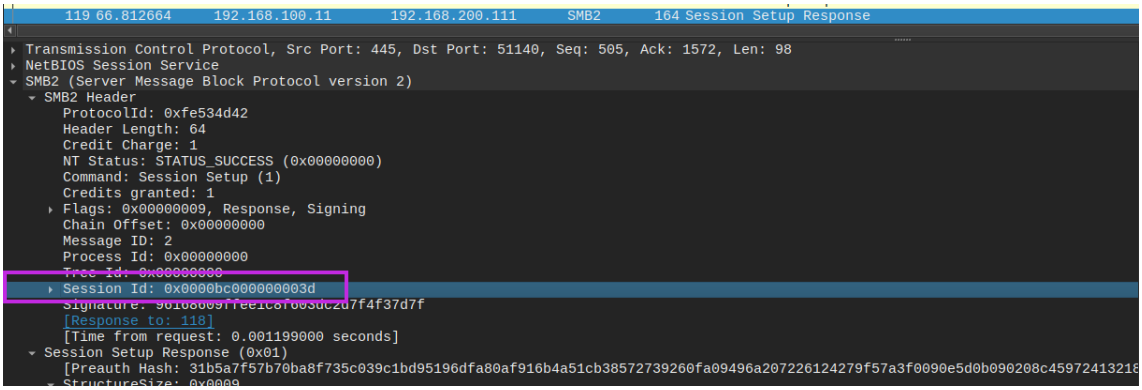
4. This allows us to obtain the following service session key from the TGS-REP: **fa1a501f6828025cc5feb7dfe1968de4**.

```
service session key: fa1a501f6828025cc5feb7dfe1968de4
```

Decryption of encrypted SMB3

1. As mentioned earlier, the service session key and service session ID are required to decrypt encrypted SMB3 packets in Wireshark. We have already obtained the service session key, so we can obtain the service session ID from the packet where the SMB session was established, No. 119.

   Service Session ID: **0000bc000000003d**

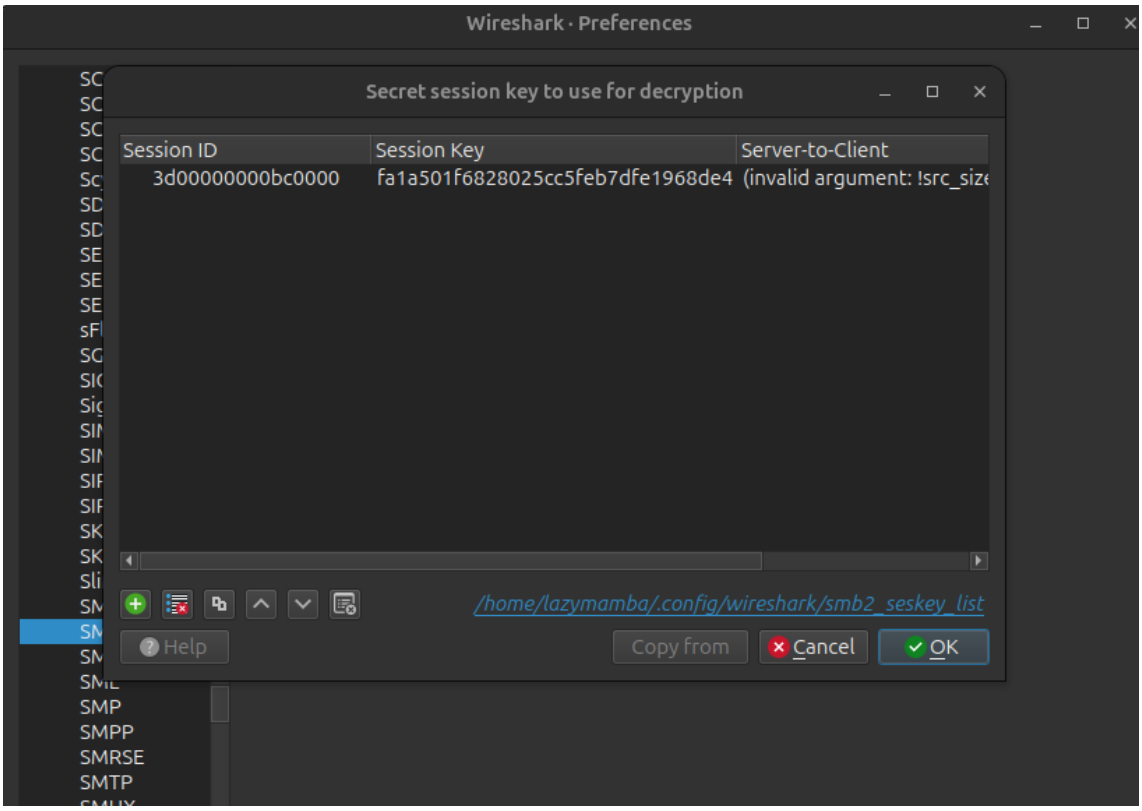


2. In Wireshark, go to **Edit > Preference > Protocols > SMB2**, select Edit in Secret session keys for decryption, and enter the service session ID and service session key.

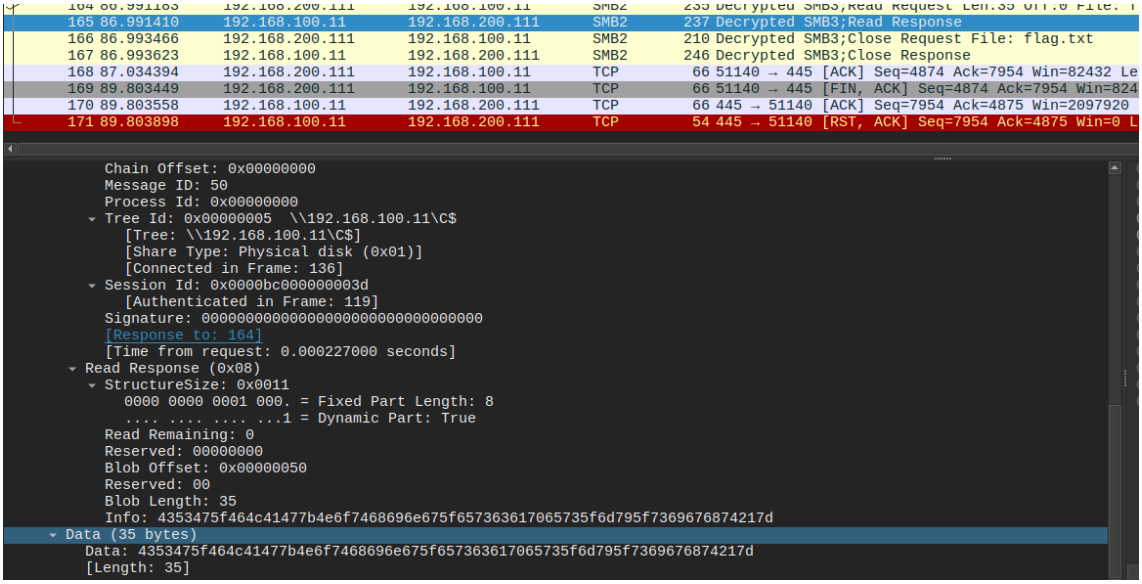
   NOTE! The Session ID obtained above was 0x0000bc000000003d, but endian conversion is required when entering it from Wireshark settings.

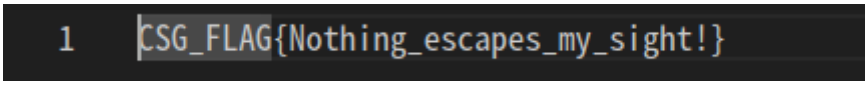
   Service Session ID: **3d00000000bc0000**

   Service Session Key: **fa1a501f6828025cc5feb7dfe1968de4**



3. You should now be able to confirm that the SMB packets have been decrypted successfully. The SMB3 Read Response contains data related to the file read by cat, etc., so right-clicking the Data field and selecting Export Packet Bytes will reveal the flag.





Flag: **CSG_FLAG{Nothing_escapes_my_sight!}**